ReliableML Method Note: Reliable Pipeline and Persistence

Jabulani Monchwe, Olorato Motitswane, and Mfundo Monchwe

Version 0.1.1

Abstract

`ReliablePipeline` retains the fitted estimator and a training-feature reference together. It supports inference-time drift reports, optional visualizations, labeled evaluation, and persistence.

1 Fit and predict

At fit time, the pipeline records the feature schema and reference distributions alongside fitting the estimator. At predict time, input schema is validated, predictions are produced, and the incoming feature batch is compared with the stored reference. A returned report can include feature metrics, aggregate summaries, warnings, and paths to optional plots or JSON reports. The monitoring options support callers that need to avoid retaining large reference data or producing plots; disabling monitoring removes those checks and must be an explicit decision.

2 Unlabeled versus labeled reports

Prediction-time distribution monitoring can operate without labels. Accuracy, confusion counts, MCC, fairness comparisons, and deterioration require true labels and are computed through the separate evaluation or history APIs. Distribution similarity is not a substitute for performance or calibration. The report's similarity score is descriptive, not a probability of correctness.

3 Persistence

`save()` serializes the estimator and reference profile together so the same fitted transformation is used after loading. The profile may include training feature observations used for plots and bootstrap computations; artifacts therefore require privacy protection similar to the source data. Python pickle can execute code when loading an untrusted artifact: load only files from trusted sources and use compatible dependency versions.

4 Operational checks

Use identical feature names, ordering, units, and preprocessing at training and inference. Validate reliability thresholds against representative historical and shifted samples. Monitor labeled performance when outcomes arrive. Avoid interpreting a green or low-drift report as a guarantee of correct prediction.

5 Interface

```
model = ReliablePipeline(
    estimator=classifier,
    reliability_required=True,
    plot_dir="reliability_plots",
).fit(X_train, y_train)

predictions, report = model.predict_with_report(X_new)
model.save("reliable-model.pkl")
```

6 Further reading

References

[1] ReliableML API and operational details: https://github.com/Jabualani/ML-Reliabilty.